I get the following eroors when running thist can you fixt it? I am running it in powershell .\RestoreFileTypes_Pure.bat

ERROR ^ : The term '^' is not recognized as the name of a cmdlet, function, script file, or operable program. Check the spelling of the name, or if a path was included, verify that the path is correct and try again. At line:1 char:32

- $ErrorActionPreference='Stop'; ^
- ~

  - CategoryInfo : ObjectNotFound: (^:String) [], ParentContainsErrorRecordException
  - FullyQualifiedErrorId : CommandNotFoundException

'Add-Type' is not recognized as an internal or external command, operable program or batch file. '#' is not recognized as an internal or external command, operable program or batch file. '#' is not recognized as an internal or external command, operable program or batch file. '#' is not recognized as an internal or external command, operable program or batch file. -ge was unexpected at this time.

SCRIPT RestoreFileTypes_Pure.bat

@echo off setlocal EnableExtensions

set "SCRIPT_NAME=RestoreFileTypes_Pure"

%SystemRoot%\System32\WindowsPowerShell\v1.0\powershell.exe -NoProfile -ExecutionPolicy Bypass -STA -Command ^ "$ErrorActionPreference='Stop'; ^ Add-Type -AssemblyName System.Windows.Forms; ^ Add-Type -AssemblyName System.Drawing; ^

function HasExtension([string]$p){ ^ $n=[IO.Path]::GetFileName($p); ^ return ($n -match '.[^.]+$'); ^ }; ^

function Get-ExtFromMime([string]$mime){ ^ if([string]::IsNullOrWhiteSpace($mime)){ return $null }; ^ try{ ^ $ctPath = 'Registry::HKEY_CLASSES_ROOT\MIME\Database\Content Type' + $mime; ^ $ext = (Get-ItemProperty -Path $ctPath -Name Extension -ErrorAction SilentlyContinue).Extension; ^ if($ext){ return $ext }; ^ } catch{}; ^ $map = @{ ^ # Documents (subset from prior script) ^ 'application/pdf'='.pdf'; ^ 'application/zip'='.zip'; ^ 'application/x-7z-compressed'='.7z'; ^ 'application/x-rar-compressed'='.rar'; ^ 'application/x-gzip'='.gz'; ^ 'application/x-bzip2'='.bz2'; ^ 'application/x-tar'='.tar'; ^ 'application/xml'='.xml'; ^ 'text/xml'='.xml'; ^ 'text/html'='.html'; ^ 'application/json'='.json'; ^ 'text/plain'='.txt'; ^ 'text/plain; charset=utf-8'='.txt'; ^

```
 # Office OpenXML ^
 'application/vnd.openxmlformats-officedocument.wordprocessingml.document'='.docx'; ^
 'application/vnd.openxmlformats-officedocument.spreadsheetml.sheet'='.xlsx'; ^
 'application/vnd.openxmlformats-officedocument.presentationml.presentation'='.pptx'; ^

 # Images (existing + web family) ^
 'image/png'='.png'; ^
 'image/jpeg'='.jpg'; ^
 'image/gif'='.gif'; ^
 'image/bmp'='.bmp'; ^
 'image/tiff'='.tif'; ^
 'image/webp'='.webp'; ^
 'image/avif'='.avif'; ^

 # Video (your requested list) ^
 'video/mp4'='.mp4'; ^
 'video/quicktime'='.mov'; ^
 'video/x-m4v'='.m4v'; ^
 'video/webm'='.webm'; ^
 'video/x-matroska'='.mkv'; ^
 'video/x-flv'='.flv'; ^
 'video/x-msvideo'='.avi'; ^
 'video/ogg'='.ogg'; ^
 'video/mp2t'='.ts'; ^
 'video/mpeg'='.mpg'; ^

 # Generic container fallback ^
 'application/epub+zip'='.epub'; ^
 'application/vnd.ms-office'='.doc'; ^
 'application/x-dosexec'='.exe'; ^
 'application/x-msdownload'='.exe'; ^
 'application/x-dll'='.dll' ^
}; ^
return $map[$mime] ^
```

}; ^

function Make-UniquePath([string]$targetPath){ ^ if(-not (Test-Path -LiteralPath $targetPath)){ return $targetPath }; ^ $dir = [IO.Path]::GetDirectoryName($targetPath); ^ $base = [IO.Path]::GetFileNameWithoutExtension($targetPath); ^ $ext = [IO.Path]::GetExtension($targetPath); ^ for($i=1; $i -lt 100000; $i++){ ^ $cand = [IO.Path]::Combine($dir, $base + '_' + $i + $ext); ^ if(-not (Test-Path -LiteralPath $cand)){ return $cand } ^ }; ^ return $targetPath ^ }; ^

function Read-Bytes([string]$p, [int]$count){ ^ $fs = [IO.File]::Open($p,[IO.FileMode]::Open,[IO.FileAccess]::Read,[IO.FileShare]::ReadWrite); ^ try{ ^ $buf = New-Object byte[] $count; ^ $read = $fs.Read($buf,0,$count); ^ if($read -lt $count){ $buf = $buf[0..($read-1)] }; ^ return ,$buf ^ } finally { $fs.Close() } ^ }; ^

function Is-LikelyText([string]$p){ ^ $bytes = Read-Bytes $p 8192; ^ if(-not $bytes -or $bytes.Length -lt 8){ return $false }; ^ $printable = 0; ^ foreach($b in $bytes){ ^ if(($b -ge 9 -and $b -le 13) -or ($b -ge 32 -and $b -le 126) -or ($b -eq 0x09)){ $printable++ } ^ }; ^ return (($printable / $bytes.Length) -ge 0.92) ^ }; ^

function Detect-PureMimeAndExtension([string]$p){ ^ # Read enough for video/container heuristics ^ $b = Read-Bytes $p 4096; ^ if(-not $b -or $b.Length -lt 1){ return [pscustomobject]@{mime=$null; ext=$null; reason='empty/read-fail'} } ^

```
# Helpers ^
function Ascii([int]$off,[int]$len){ ^
  if($b.Length -lt ($off+$len)){ return '' } ^
  return [Text.Encoding]::ASCII.GetString($b,$off,$len) ^
}; ^
```

```
# PDF ^
if($b.Length -ge 5 -and $b[0..4] -ceq ( [byte[]](0x25,0x50,0x44,0x46,0x2D) )){ ^
  $mime='application/pdf'; return [pscustomobject]@{mime=$mime; ext=(Get-ExtFromMime $mime); reason='PDF signature'} } ^

# ZIP (PK..) ^
if($b.Length -ge 4 -and $b[0]-eq 0x50 -and $b[1]-eq 0x4B){ ^
  try{ ^
    $fs = [IO.File]::Open($p,[IO.FileMode]::Open,[IO.FileAccess]::Read,[IO.FileShare]::ReadWrite); ^
    try{ ^
      $za = New-Object System.IO.Compression.ZipArchive($fs,[System.IO.Compression.ZipArchiveMode]::Read,$false); ^
      $has = New-Object System.Collections.Generic.HashSet[string]; ^
      foreach($e in $za.Entries){ [void]$has.Add($e.FullName) } ^
      if($has.Contains('mimetype')){ ^
        $mime='application/epub+zip'; ^
      } else { ^
        if($has.Contains('word/document.xml')){ $mime='application/vnd.openxmlformats-officedocument.wordprocessingml.document' } ^
        elseif($has.Contains('xl/workbook.xml')){ $mime='application/vnd.openxmlformats-officedocument.spreadsheetml.sheet' } ^
        elseif($has.Contains('ppt/presentation.xml')){ $mime='application/vnd.openxmlformats-officedocument.presentationml.presentation' } ^
        else { $mime='application/zip' } ^
      } ^
      $ext = Get-ExtFromMime $mime; if(-not $ext){ $ext='.zip' }; ^
      return [pscustomobject]@{mime=$mime; ext=$ext; reason='ZIP signature + entry inspection'} ^
    } finally { $fs.Close() } ^
  } catch { ^
    $mime='application/zip'; return [pscustomobject]@{mime=$mime; ext='.zip'; reason='ZIP signature (no entry inspection)'} ^
  } ^
}; ^

# 7z ^
if($b.Length -ge 6 -and $b[0]-eq 0x37 -and $b[1]-eq 0x7A -and $b[2]-eq 0xBC -and $b[3]-eq 0xAF -and $b[4]-eq 0x27 -and $b[5]-eq 0x1C){ ^
  $mime='application/x-7z-compressed'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.7z' }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='7z signature'} ^
}; ^

# RAR ^
if($b.Length -ge 8 -and $b[0]-eq 0x52 -and $b[1]-eq 0x61 -and $b[2]-eq 0x72 -and $b[3]-eq 0x21 -and $b[4]-eq 0x1A){ ^
  $mime='application/x-rar-compressed'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.rar' }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='RAR signature'} ^
}; ^

# GZIP ^
if($b.Length -ge 2 -and $b[0]-eq 0x1F -and $b[1]-eq 0x8B){ ^
  $mime='application/x-gzip'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.gz' }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='GZIP signature'} ^
}; ^

# BZIP2 ^
if($b.Length -ge 3 -and $b[0]-eq 0x42 -and $b[1]-eq 0x5A -and $b[2]-eq 0x68){ ^
  $mime='application/x-bzip2'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.bz2' }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='BZIP2 signature'} ^
}; ^

# ZSTD ^
if($b.Length -ge 4 -and $b[0]-eq 0x28 -and $b[1]-eq 0xB5 -and $b[2]-eq 0x2F -and $b[3]-eq 0xFD){ ^
  $mime='application/zstd'; $ext='.zst'; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='ZSTD signature'} ^
}; ^

# PNG ^
if($b.Length -ge 8 -and ($b[0]-eq 0x89 -and $b[1]-eq 0x50 -and $b[2]-eq 0x4E -and $b[3]-eq 0x47 -and $b[4]-eq 0x0D -and $b[5]-eq 0x0A -and $b[6]-eq 0x1A -and $b[7]-eq 0x0A)){ ^
  $mime='image/png'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.png' }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='PNG signature'} ^
}; ^

# JPEG ^
if($b.Length -ge 3 -and $b[0]-eq 0xFF -and $b[1]-eq 0xD8 -and $b[2]-eq 0xFF){ ^
  $mime='image/jpeg'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.jpg' }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='JPEG signature'} ^
}; ^

# GIF ^
if($b.Length -ge 6){ ^
  $s = [Text.Encoding]::ASCII.GetString($b,0,6); ^
  if($s -eq 'GIF87a' -or $s -eq 'GIF89a'){ ^
    $mime='image/gif'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.gif' }; ^
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='GIF signature'} ^
  } ^
}; ^

# BMP ^
if($b.Length -ge 2 -and $b[0]-eq 0x42 -and $b[1]-eq 0x4D){ ^
  $mime='image/bmp'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.bmp' }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='BMP signature'} ^
}; ^

# TIFF ^
if($b.Length -ge 4){ ^
  if(($b[0]-eq 0x49 -and $b[1]-eq 0x49 -and $b[2]-eq 0x2A -and $b[3]-eq 0x00) -or ($b[0]-eq 0x4D -and $b[1]-eq 0x4D -and $b[2]-eq 0x00 -and $b[3]-eq 0x2A)){ ^
    $mime='image/tiff'; $ext=Get-ExtFromMime $mime; if(-not $ext){ $ext='.tif' }; ^
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='TIFF signature'} ^
  } ^
}; ^

# OLE compound (generic fallback) ^
if($b.Length -ge 8 -and $b[0]-eq 0xD0 -and $b[1]-eq 0xCF -and $b[2]-eq 0x11 -and $b[3]-eq 0xE0 -and $b[4]-eq 0xA1 -and $b[5]-eq 0xB1 -and $b[6]-eq 0x1A -and $b[7]-eq 0xE1){ ^
  $mime='application/vnd.ms-office'; $ext='.doc'; ^
```

```
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='OLE compound signature (MS Office generic)'} ^
}; ^

# PE (exe/dll) ^
if($b.Length -ge 2 -and $b[0]-eq 0x4D -and $b[1]-eq 0x5A){ ^
  $mime='application/x-dosexec'; $ext='.exe'; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='PE signature (MZ)'} ^
}; ^

# ==========================
# Video + media additions
# ==========================

# TS (MPEG-TS): sync byte 0x47 at 0,188,376...
if($b.Length -ge 564){ ^
  if($b[0]-eq 0x47 -and $b[188]-eq 0x47 -and $b[376]-eq 0x47){ ^
    $mime='video/mp2t'; $ext='.ts'; ^
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='TS sync-byte heuristic'} ^
  } ^
} ^
# MPEG program stream: 00 00 01 BA or B3
if($b.Length -ge 12){ ^
  if($b[0]-eq 0x00 -and $b[1]-eq 0x00 -and $b[2]-eq 0x01){ ^
    if($b[3]-eq 0xBA){ $mime='video/mpeg'; $ext='.mpg'; return [pscustomobject]@{mime=$mime; ext=$ext; reason='MPEG pack header'} } ^
    if($b[3]-eq 0xB3){ $mime='video/mpeg'; $ext='.mpg'; return [pscustomobject]@{mime=$mime; ext=$ext; reason='MPEG system/stream header'} } ^
  } ^
}; ^

# OGG: OggS
if($b.Length -ge 4 -and ($b[0..3] -ceq ([byte[]](0x4F,0x67,0x67,0x53)))){ ^
  $mime='video/ogg'; $ext='.ogg'; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='OggS signature'} ^
}; ^

# FLV: "FLV" 0x01
if($b.Length -ge 4){ ^
  if( (Ascii 0 3) -eq 'FLV' -and $b[3] -eq 0x01 ){ ^
    $mime='video/x-flv'; $ext='.flv'; ^
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='FLV signature'} ^
  } ^
}; ^

# RIFF family: AVI / WebP
if($b.Length -ge 12 -and (Ascii 0 4) -eq 'RIFF'){ ^
  $tag = Ascii 8 4; ^
  if($tag -eq 'AVI '){ ^
    $mime='video/x-msvideo'; $ext='.avi'; ^
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='RIFF AVI signature'} ^
  } elseif($tag -eq 'WEBP'){ ^
    $mime='image/webp'; $ext='.webp'; ^
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='RIFF WebP signature'} ^
  } ^
}; ^

# EBML (MKV/WebM): 1A 45 DF A3 ...
if($b.Length -ge 4 -and $b[0]-eq 0x1A -and $b[1]-eq 0x45 -and $b[2]-eq 0xDF -and $b[3]-eq 0xA3){ ^
  # Guess WebM by searching for 'webm' string early
  $text = ''; ^
  try { $text = [Text.Encoding]::ASCII.GetString($b,0,[Math]::Min(2048,$b.Length)) } catch{}; ^
  if($text -match 'webm'){ ^
    $mime='video/webm'; $ext='.webm'; ^
  } else { ^
    $mime='video/x-matroska'; $ext='.mkv'; ^
  }; ^
  return [pscustomobject]@{mime=$mime; ext=$ext; reason='EBML signature (MKV/WebM)'} ^
}; ^

# ISO BMFF ftyp parsing: MP4/MOV/M4V/AVIF
# MP4 family typically has: [size][ftyp][major_brand(4)][minor_version(4)] at offset 4..15
if($b.Length -ge 16){ ^
  $ftyp = Ascii 4 4; ^
  if($ftyp -eq 'ftyp'){ ^
    $major = Ascii 8 4; ^
    # AVIF
    if($major -match 'avif|avis'){ ^
      $mime='image/avif'; $ext='.avif'; ^
      return [pscustomobject]@{mime=$mime; ext=$ext; reason='BMFF ftyp major-brand (AVIF)'} ^
    } ^
    # MOV commonly uses 'qt  ' major brand
    if($major -match '^qt'){ ^
      $mime='video/quicktime'; $ext='.mov'; ^
      return [pscustomobject]@{mime=$mime; ext=$ext; reason='BMFF ftyp major-brand (MOV/qt)'} ^
    } ^
    # M4V sometimes uses 'M4V ' major brand
    if($major -match '^M4V'){ ^
      $mime='video/x-m4v'; $ext='.m4v'; ^
      return [pscustomobject]@{mime=$mime; ext=$ext; reason='BMFF ftyp major-brand (M4V)'} ^
    } ^
    # MP4 family brands
    if($major -in @('isom','iso2','mp41','mp42','avc1','MSNV')){ ^
      $mime='video/mp4'; $ext='.mp4'; ^
      return [pscustomobject]@{mime=$mime; ext=$ext; reason='BMFF ftyp major-brand (MP4 family)'} ^
    } ^
    # If brand is unknown, still treat as mp4 base
    # but keep extension conservative.
    $mime='video/mp4'; $ext='.mp4'; ^
```

```
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='BMFF ftyp (assumed MP4)'} ^
  } ^
}; ^

# ==========================
# Text-ish heuristics (keep last)
# ==========================
if(Is-LikelyText $p){ ^
  try{ ^
    $txt = [IO.File]::ReadAllText($p); ^
    $t = $txt.Trim(); ^
    if($t.StartsWith('<!DOCTYPE') -or $t.StartsWith('<html') -or $t -match '<head' -or $t -match '<body'){ ^
      $mime='text/html'; $ext='.html'; return [pscustomobject]@{mime=$mime; ext=$ext; reason='text/html heuristic'} ^
    } ^
    if($t.StartsWith('<?xml') -or $t -match '^<[^>]+:'){ ^
      $mime='application/xml'; $ext='.xml'; return [pscustomobject]@{mime=$mime; ext=$ext; reason='XML heuristic'} ^
    } ^
    if($t -match '^\{[\s\S]*\}$' -or $t -match '^\[[\s\S]*\]$'){ ^
      $mime='application/json'; $ext='.json'; return [pscustomobject]@{mime=$mime; ext=$ext; reason='JSON heuristic'} ^
    } ^
    if($t.StartsWith('#!')){ ^
      if($t -match 'python'){ $mime='text/x-script.python'; $ext='.py' } ^
      elseif($t -match 'sh|bash'){ $mime='text/x-shellscript'; $ext='.sh' } ^
      else{ $mime='text/plain'; $ext='.txt' } ^
      return [pscustomobject]@{mime=$mime; ext=$ext; reason='shebang heuristic'} ^
    } ^
    $mime='text/plain'; $ext='.txt'; ^
    return [pscustomobject]@{mime=$mime; ext=$ext; reason='plain-text heuristic'} ^
  } catch{} ^
}; ^

return [pscustomobject]@{mime=$null; ext=$null; reason='unknown/no signature matched'} ^


}; ^
```

GUI

```
$form = New-Object System.Windows.Forms.Form; ^ $form.Text = '$SCRIPT_NAME - Magic detection + extension restore'; ^ $form.Size = New-Object System.Drawing.Size(700,430); ^ $form.StartPosition = 'CenterScreen'; ^ $form.Topmost = $true; ^ ^ $lbl = New-Object System.Windows.Forms.Label; ^ $lbl.Location = New-Object System.Drawing.Point(10,15); ^ $lbl.Size = New-Object System.Drawing.Size(650,20); ^ $lbl.Text = 'Select the root folder that contains recovered “no extension” files.'; ^ $form.Controls.Add($lbl); ^ ^ $txtRoot = New-Object System.Windows.Forms.TextBox; ^ $txtRoot.Location = New-Object System.Drawing.Point(10,40); ^ $txtRoot.Size = New-Object System.Drawing.Size(560,25); ^ $form.Controls.Add($txtRoot); ^ ^ $btnBrowse = New-Object System.Windows.Forms.Button; ^ $btnBrowse.Location = New-Object System.Drawing.Point(580,38); ^ $btnBrowse.Size = New-Object System.Drawing.Size(95,28); ^ $btnBrowse.Text = 'Browse...'; ^ $form.Controls.Add($btnBrowse); ^ ^ $rbReport = New-Object System.Windows.Forms.RadioButton; ^ $rbReport.Location = New-Object System.Drawing.Point(10,85); ^ $rbReport.Size = New-Object System.Drawing.Size(230,20); ^ $rbReport.Text = 'Report only (no rename)'; ^ $rbReport.Checked = $true; ^ $form.Controls.Add($rbReport); ^ ^ $rbCopy = New-Object System.Windows.Forms.RadioButton; ^ $rbCopy.Location = New-Object System.Drawing.Point(250,85); ^ $rbCopy.Size = New-Object System.Drawing.Size(260,20); ^ $rbCopy.Text = 'Create copy WITH extension (leave original)'; ^ $form.Controls.Add($rbCopy); ^ ^ $rbRename = New-Object System.Windows.Forms.RadioButton; ^ $rbRename.Location = New-Object System.Drawing.Point(10,110); ^ $rbRename.Size = New-Object System.Drawing.Size(470,20); ^ $rbRename.Text = 'Rename in place (append extension)'; ^ $form.Controls.Add($rbRename); ^ ^ $chkDry = New-Object System.Windows.Forms.CheckBox; ^ $chkDry.Location = New-Object System.Drawing.Point(490,108); ^ $chkDry.Size = New-Object System.Drawing.Size(190,20); ^ $chkDry.Text = 'Dry run (no file changes)'; ^ $form.Controls.Add($chkDry); ^ ^ $chkOnlyNoExt = New-Object System.Windows.Forms.CheckBox; ^ $chkOnlyNoExt.Location = New-Object System.Drawing.Point(10,135); ^ $chkOnlyNoExt.Size = New-Object System.Drawing.Size(250,20); ^ $chkOnlyNoExt.Checked = $true; ^ $chkOnlyNoExt.Text = 'Only process files with NO extension'; ^ $form.Controls.Add($chkOnlyNoExt); ^ ^ $progress = New-Object System.Windows.Forms.ProgressBar; ^ $progress.Location = New-Object System.Drawing.Point(10,165); ^ $progress.Size = New-Object System.Drawing.Size(665,20); ^ $form.Controls.Add($progress); ^ ^ $lst = New-Object System.Windows.Forms.ListBox; ^ $lst.Location = New-Object System.Drawing.Point(10,195); ^ $lst.Size = New-Object System.Drawing.Size(665,170); ^ $form.Controls.Add($lst); ^ ^ $btnStart = New-Object System.Windows.Forms.Button; ^ $btnStart.Location = New-Object System.Drawing.Point(10,375); ^ $btnStart.Size = New-Object System.Drawing.Size(110,30); ^ $btnStart.Text = 'Start'; ^ $form.Controls.Add($btnStart); ^ ^ $btnClose = New-Object System.Windows.Forms.Button; ^ $btnClose.Location = New-Object System.Drawing.Point(120,375); ^ $btnClose.Size = New-Object System.Drawing.Size(110,30); ^ $btnClose.Text = 'Close'; ^ $form.Controls.Add($btnClose); ^ ^ $btnClose.Add_Click({ $form.Close() }); ^ $btnBrowse.Add_Click({ ^ $fd = New-Object System.Windows.Forms.FolderBrowserDialog; ^ $fd.ShowNewFolderButton = $false; ^ if($fd.ShowDialog() -eq [System.Windows.Forms.DialogResult]::OK){ $txtRoot.Text = $fd.SelectedPath } ^ }); ^ ^ function Get-Mode(){ if($rbRename.Checked){ 'rename' } elseif($rbCopy.Checked){ 'copy' } else { 'report' } } ^ ^ $btnStart.Add_Click({ ^ $root = $txtRoot.Text.Trim(); ^ if([string]::IsNullOrWhiteSpace($root) -or -not (Test-Path -LiteralPath $root)){ [System.Windows.Forms.MessageBox]::Show('Pick a valid folder.'); return } ^ $progress.Value = 0; $lst.Items.Clear(); ^ $mode = Get-Mode; ^ $dry = $chkDry.Checked; ^ $onlyNoExt = $chkOnlyNoExt.Checked; ^ $reportPath = Join-Path $root 'mime-restore-report.csv'; ^ 'path,mime,extension,reason,action' | Out-File -FilePath $reportPath -Encoding UTF8; ^ $files = Get-ChildItem -LiteralPath $root -Recurse -File; ^ if($onlyNoExt){ $files = $files | Where-Object { -not (HasExtension $_.FullName) } } ^ $total = $files.Count; ^ if($total -eq 0){ [System.Windows.Forms.MessageBox]::Show('No matching files found.'); return } ^ $i=0; ^ foreach($f in $files){ ^ $i++; ^ $pct = int; ^ $progress.Value = [Math]::Max(0,[Math]::Min(100,$pct)); ^ $res = Detect-PureMimeAndExtension $f.FullName; ^ $mime = $res.mime; ^ $ext = $res.ext; ^ $reason = $res.reason; ^ $action = ''; ^ if(-not $ext){ ^ if(-not [string]::IsNullOrWhiteSpace($mime)){ $action = $mode } else { $action = 'unknown-no-ext' } ^ } else { ^ if($mode -eq 'report'){ $action='report' } ^ elseif($dry){ $action='dry-run-no-change' } ^ else { ^ $target = [IO.Path]::Combine([IO.Path]::GetDirectoryName($f.FullName), [IO.Path]::GetFileName($f.FullName) + $ext); ^ $target = Make-UniquePath $target; ^ if($mode -eq 'copy'){ ^ Copy-Item -LiteralPath $f.FullName -Destination $target -Force; ^ $action='copy-to:' + (Split-Path $target -Leaf); ^ } elseif($mode -eq 'rename'){ ^ Rename-Item -LiteralPath $f.FullName -NewName ([IO.Path]::GetFileName($target)); ^ $action='rename-to:' + (Split-Path $target -Leaf); ^ } ^ } ^ } ^ $line = ('"{0}","{1}","{2}","{3}","{4}"' -f $f.FullName.Replace('"','""'), ($mime -replace '"',''), ($ext -replace '"',''), ($reason -replace '"',''), ($action -replace '"','')); ^ Add-Content -Path $reportPath -Value $line -Encoding UTF8; ^ if($lst.Items.Count -gt 3000){ $lst.Items.RemoveAt(0) } ^ $lst.Items.Add(($f.Name + ' => ' + ($mime ?? 'unknown') + ' ' + ($ext ?? ''))) | Out-Null; ^ $form.Refresh(); ^ }; ^ [System.Windows.Forms.MessageBox]::Show('Done. Report: ' + $reportPath); ^ }); ^ ^ $form.Add_Shown({ $form.Activate() }); ^ [void] $form.ShowDialog() "

echo. echo Done. exit /b 0
```